

QA OS User Manual

QA OS turns Claude into a QA assistant for our applications. You talk to it in plain English in the Claude desktop app, and use a few **slash commands** that start with `/qa-os:`.

Setup: `docs/TRAINING_GUIDE.md` section 1. Open the folder `<drive>:\qa-os` in Claude, then type `/qa-os:` in the message box to see the list.

The commands at a glance

COMMAND	WHAT IT DOES	USE IT WHEN
<code>/qa-os:status</code>	Shows recent runs and what Claude knows about each app	Start of the day, or to check progress
<code>/qa-os:train</code>	Training session: teach Claude the business	Now (S&D training phase)
<code>/qa-os:quick</code>	Short request or one line → Claude's own steps → live run → Regress Master SQL + Excel	Main use after training
<code>/qa-os:learn</code>	Refreshes screens, menus and field labels of an app	After a release changes screens
<code>/qa-os:cases</code>	Jira story → requirement analysis → test cases + Excel	Only if a Jira story is available
<code>/qa-os:steps</code>	Writes executable test steps for those cases	After cases are approved
<code>/qa-os:run</code>	Does cases and steps in one go for a story	Only if a Jira story is available
<code>/qa-os:play</code>	Checks a saved flow and tells you how to run it without AI	To replay a finished flow

After training, QA OS is used through `/qa-os:quick` with short requests. The Jira-based commands (`cases` , `steps` , `run`) remain available but are not the normal starting point.

`/qa-os:status` Where are we?

```
/qa-os:status
/qa-os:status SDMS-2990
```

You get: the latest runs (ticket or case, verdict, failed step), and for each app how many screens are known and when they were last harvested.

/qa-os:train Teach Claude the business

```
/qa-os:train snd  
/qa-os:train snd PK walk 11  
/qa-os:train snd BD explain
```

Claude first asks your **name**, the **market** (PK or BD), the **environment** (cnr1dev1) and how you want to train today. Then use any of these methods.

a) Give it a document

Copy the file into `apps\snd\knowledge\sources\inbox\` , then:

YOU

I put "Route Settlement user guide.pdf" in the inbox. Learn it and tell me what is different from what you already know.

CLAUDE

I added 14 rules from chapters 3–5 to the settlement page. Two of them conflict with what I saw live on 6 Oct. Which is right: ...?

b) Explain in your own words

YOU

In Bangladesh, a CLP incentive creates a credit note at month end. It is adjusted against the outlet's next invoice.

CLAUDE

Recorded as stated by you today. It also answers open question Q-...; I've updated the delivery and returns page.

c) Ask it to run a framework group live

YOU

Execute group 11 for PK today from seq 1.

CLAUDE

Starting group 11 (46 active steps). Please log in as Auto_Multi_Orga in the browser window.

YOU

Logged in, go ahead.

At each user switch Claude logs out and tells you who to log in as. **You type the user and password in the browser.** Claude can press the Login button for you, and it picks the company and distributor itself. If it gets stuck, it stops and asks you.

d) Questions and quiz

YOU

Show me the open questions for settlement.

YOU

I'll give you a test case: predict the result first, then execute it.

Finish

YOU

Close the training session.

CLAUDE

Merged into the knowledge pages, wrote the session report and updated STATUS.md.
Questions answered: ... Still open: ...

Then send your work back: see the training guide, section 6.

/qa-os:quick The main way to work after training (no ticket)

```
/qa-os:quick snd "create one order and order detail with two products"  
/qa-os:quick snd "BD: book an order for one outlet with three SKUs, then allocate it"
```

You give a short request or one line; Claude does the rest:

1. **Understands it from its training.** It maps your words to the business actions and screens it was trained on. If a screen hasn't been trained yet, it stops and asks you to train it first (`/qa-os:train`).
2. **Writes its own test steps.** The steps, rules and expected messages come from what you taught it, not from the old Selenium framework workbooks or flows.
3. **Picks today's data** from the database: an active outlet, and SKUs that are in stock.
4. **You approve once.** Claude then executes the flow live a single time. You type the logins.
5. **Generates the Regress Master scripts:** `framework.sql` (the new test flow, its screens, fields and events), a rollback script, and the Excel case data, all checked by the verifier.

YOU

/qa-os:quick snd "create one order and order detail with two products"

CLAUDE

From my training: Order Booking (Maker). Here are my steps and today's data (outlet 1000000004, SKUs 62740537 × 2 CS and 20050310 × 5 CS). Approve?

YOU

Approved.

CLAUDE

Executed: order COL2600000xxxx, "Order Save successfully". Generated framework.sql, framework_rollback.sql and the Excel case data. The framework owner applies the SQL on a test copy first.

/qa-os:learn Refresh screen knowledge

```
/qa-os:learn snd  
/qa-os:learn snd "Transaction > Receivable > Deposit Slip"
```

You get: what was added or changed (menus, fields, labels), and any differences between the database definitions and the live screens.

/qa-os:cases From a Jira story to test cases OPTIONAL · JIRA STORY

```
/qa-os:cases SDMS-2990  
/qa-os:cases SDMS-2990 --story C:\temp\SDMS-2990.docx
```

The Jira connector must be signed in; otherwise give `--story` with an exported or pasted story.

1. Claude analyses the story: rules, acceptance criteria, and **ambiguities**. Answer the ambiguities (gate G1).
2. It designs up to 10 core cases plus a backlog, and writes the team Excel workbook.
3. You review and approve the cases (gate G2). Reply "approved", or give your changes.

/qa-os:steps Write the test steps AFTER CASES ARE APPROVED

```
/qa-os:steps SDMS-2990
```

Claude writes the steps in our standard wording, for example:

```
[Maker] Navigate to Dispatch Advice  
[Maker] Create Dispatch Advice with Warehouse = Auto Main Warehouse  
[Checker] Forward Dispatch Advice with comment "Automation Approval"
```

It checks each label against the real screen and marks every step **clear** or **needs input**. Fix the "needs input" steps in the Excel (column *Test Steps*, plain English), then approve (gate G3).

`/qa-os:run` The whole front half in one command OPTIONAL · JIRA STORY

```
/qa-os:run SDMS-2990 --app snd --env cnr1dev1
```

This runs analysis → cases → steps, stopping at each gate for your approval. At the end you get the workbook path and the list of cases ready to run.

`/qa-os:play` Replay a finished flow without AI

```
/qa-os:play runs/SDMS-2990/20261007-1037/flows/TC01.json
```

Claude dry-runs the flow first. If the flow needs a login, it gives you the command to run yourself, so your password never goes through Claude:

```
python runtime/qaos_player.py runs/SDMS-2990/20261007-1037/flows/TC01.json
```

When it finishes, say "done". Claude summarises the result and the evidence files.

Good to know

- **Claude never types passwords** and never applies SQL. Database access is read-only.
- **Gates:** Claude waits for your approval at each gate. "approved", "go ahead", "yes" or "LGTM" approve; anything else is treated as feedback.
- **Plain English works too.** You don't have to use commands; for example, "what are the open questions for Order Editing?" or "why did Route Settlement fail?"
- **One daily cycle per distributor per day** on the shared environment.
- **Stuck?** Look at the troubleshooting table in the training guide (section 8), or ask Claude: "check docs/STATUS.md and tell me where we are".